

System Monitorowania Jakości Danych Pogodowych

Raport Końcowy z Projektu

Michał Proc 198407, Maciej Skibiński 198247

24.06.2026

Spis treści

1	Wstęp i Cel Projektu	2
2	Architektura Systemu	2
2.1	Pobieranie API i Magazynowanie Surowych Danych	2
2.2	Silnik Walidacyjny i Ocena Jakości	3
2.3	Mechanizm Kwarantanny i Raportowania	3
3	Założenia Systemu	4
3.1	Zdefiniowane Kryteria Poprawności	4
3.2	Znane Ograniczenia	5
4	Analiza Danych	5
4.1	Przegląd Zidentyfikowanych Anomalii	5
4.2	Zestawienie Ilościowe Zdiagnozowanych Błędów	5
4.3	Dynamika Przyrostu Błędów	6
5	Podsumowanie	6

1 Wstęp i Cel Projektu

Raport końcowy ma na celu opis autorskiego systemu do weryfikacji i kontroli jakości danych meteorologicznych. Założeniem projektu było stworzenie niezawodnego mechanizmu, który w stałych odstępach czasu zbiera dane ze stacji meteorologicznej GDN_01 za pośrednictwem interfejsu REST API, zapisuje surowe dane, po czym ocenia ich poprawność i zapisuje błędne wyniki.

Zaprojektowany algorytm na bieżąco oddziela poprawne obserwacje od zakłóceń sprzętowych, automatycznie sugerując optymalne techniki w przypadku zamiaru implementacji danych do dalszych systemów analitycznych. Przekształcenie surowego zbioru danych na zbiór jedynie poprawnych danych jest fundamentalnym warunkiem poprawnego działania systemów klasyfikacji i predykcji.

2 Architektura Systemu

Rozwiązanie bazuje na programie napisanym w języku Python. Cały proces przetwarzania napływających informacji podzielono na następujące etapy:

2.1 Pobieranie API i Magazynowanie Surowych Danych

Do komunikacji ze źródłem danych (REST API) użyto biblioteki `requests`. W celu zachowania pełnej historii zebranych danych, przed jakąkolwiek modyfikacją lub czyszczeniem, każdy odebrany z serwera pakiet JSON jest natychmiast dopisywany do pliku `raw_data_GDN_01.jsonl`.

```
1 def fetch_api(endpoint: str, params: dict) -> dict:
2     url = f"{API_BASE_URL}{endpoint}"
3     resp = requests.get(url, headers=HEADERS, params=params)
4     resp.raise_for_status()
5     return resp.json()
6
7 def save_to_jsonl(filename: str, data: dict):
8     with open(filename, 'a', encoding='utf-8') as f:
9         f.write(json.dumps(data) + "\n")
10
11 # Wywołanie wewnątrz głównej petli programu:
12 raw_data = fetch_api("/latest", {"station_id": STATION_ID})
13 save_to_jsonl(f"raw_data_{STATION_ID}.jsonl", raw_data)
```

Listing 1: Pobieranie danych z REST API i zapis do pliku w formacie JSONL.

```
1 {
2     "timestamp": "2026-06-14T09:13:50.173730+00:00",
3     "station_id": "GDN_01",
4     "temperature": 13.73,
5     "humidity": 83.0,
6     "pressure": 998.94,
7     "wind_speed": 0.68,
8     "wind_direction": 94,
9     "rain_mm": 0.37,
10    "cloud_cover": 33
11 }
```

Listing 2: Struktura surowego odczytu ze stacji GDN_01.

2.2 Silnik Walidacyjny i Ocena Jakości

Głównym elementem systemu jest klasa `DataQualityValidator`. Działa ona w oparciu o pamięć podręczną, przechowując w buforze 10 ostatnich stanów pogody, co umożliwia wykrywanie anomalii w czasie. Metoda `validate_record` na bieżąco analizuje każdy nowy odczyt pod kątem:

- **Występowania braków:** Wyszukiwanie pustych wartości typu `Null` w kluczowych polach.
- **Ograniczeń fizycznych:** Upewnienie się, że odczyty mieszczą się w dopuszczalnym przedziale (np. procentowe zachmurzenie między 0 a 100%).
- **Ciągłości i stabilności zjawisk:** Śledzenie nienaturalnych skoków wartości pomiędzy kolejnymi pomiarami.

```
1 def validate_record(self, current: WeatherRecord) -> List[str]:
2     errors = []
3
4     # 1. Występowanie braków
5     if self._is_empty(current.temperature):
6         errors.append("Brak danych: temperature")
7
8     # 2. Ograniczenia fizyczne (zakresy)
9     if not (-50 <= current.temperature <= 50):
10        errors.append(f"Temperature {current.temperature} poza zakresem")
11
12    if not (0 <= current.humidity <= 100):
13        errors.append(f"Humidity {current.humidity} poza zakresem")
14
15    # 3. Ciągłość i stabilność zjawisk (skoki wartości)
16    if self.history:
17        prev = self.history[-1]
18
19        if abs(current.temperature - prev.temperature) > 10:
20            errors.append("Skok Temp: Różnica > 10C")
21        if abs(current.humidity - prev.humidity) > 30:
22            errors.append("Skok Hum: Różnica > 30%")
23
24        if self._check_stuck('temperature', 10, current.temperature):
25            errors.append("Stuck sensor: Temp (10)")
26
27    self._update_history(current)
28    return errors
```

Listing 3: Fragment metody walidującej parametry pogodowe w klasie `DataQualityValidator`.

2.3 Mechanizm Kwarantanny i Raportowania

Kiedy system natrafi na uszkodzony odczyt, działanie programu nie jest przerywane. Zamiast tego, nielogiczny rekord trafia do wyizolowanego pliku `quarantine_invalid_GDN_01.jsonl`. Równolegle, tworzony jest zapis zawierający procentową ocenę jakości danego pomiaru (tzw. *Quality Score*), informujący o tym, ile parametrów w danym odczycie było poprawnych.

```

1 {
2   "raw_record": {
3     "timestamp": "2026-06-13T19:42:58.552233+00:00",
4     "station_id": "GDN_01",
5     "temperature": 19.96,
6     "humidity": 45.7,
7     "pressure": 1024.02,
8     "wind_speed": 10.75,
9     "wind_direction": 82,
10    "rain_mm": 2.82,
11    "cloud_cover": 25
12  },
13  "errors": [
14    "Skok Hum: Różnica > 30%"
15  ],
16  "cleaning_recommendations": [
17    "Zmien błędna wartość na Null, a następnie użyj interpolacji
18    liniowej do wygładzenia wykresu."
19  ],
20  "quality_score_percent": 85.71428571428571

```

Listing 4: Przykład odrzuconego rekordu w pliku kwarantanny.

```

1 {"timestamp": "2026-06-13T20:13:00.423022+00:00", "status": "rekord
   poprawny", "quality_score_percent": 100.0}
2 {"timestamp": "2026-06-13T20:28:01.555132+00:00", "status": "rekord
   poprawny", "quality_score_percent": 100.0}
3 {"timestamp": "2026-06-13T20:43:02.331571+00:00", "status": "błąd", "
   quality_score_percent": 85.71428571428571}
4 {"timestamp": "2026-06-13T20:58:03.374957+00:00", "status": "błąd", "
   quality_score_percent": 85.71428571428571}
5 {"timestamp": "2026-06-13T21:13:04.420223+00:00", "status": "błąd", "
   quality_score_percent": 85.71428571428571}
6 {"type": "cumulative_summary", "timestamp": "2026-06-13T23
   :13:05.418615", "total_analyzed_from_start": 20, "
   total_errors_from_start": 9}

```

Listing 5: Struktura zapisu raportu jakości odczytów oraz podsumowanie kumulatywne.

3 Założenia Systemu

3.1 Zdefiniowane Kryteria Poprawności

Aplikacja weryfikuje 7 kluczowych atrybutów pogodowych. Do najważniejszych reguł śledzących spójność w czasie należą:

- **Fluktuacje ciśnienia:** Skok ciśnienia atmosferycznego o ponad 15 hPa w ciągu 15 minut jest traktowany jako ewidentny błąd czujnika.
- **Niestabilność wilgotności:** Zmiana poziomu wilgotności przekraczająca 30% pomiędzy sąsiadującymi odczytami natychmiast wyzwala flagę ostrzegawczą.

3.2 Znane Ograniczenia

Aktualna implementacja wykorzystuje pamięć operacyjną do przechowywania bufora historycznego. Oznacza to, że w przypadku awarii lub zrestartowania programu, system musi najpierw jednorazowo odpytać punkt końcowy API /batch, aby zrekonstruować 10 ostatnich odczytów i przywrócić kontekst niezbędny do walidacji.

4 Analiza Danych

W ramach testów uruchomiono aplikację w trybie ciągłego nasłuchu. Zgromadzono próbkę składającą się z 70 kolejnych pomiarów ze stacji GDN_01.

4.1 Przegląd Zidentyfikowanych Anomalii

Na 70 przeanalizowanych rekordów, system oflagował problemy z jakością danych aż w 31 przypadkach. Taki wynik wskazuje na błąd w działaniu aparatury meteorologicznej lub na zastosowanie za bardzo rygorystycznych kryteriów przy sprawdzaniu błędów

Zdecydowanie najczęściej obserwowanym zjawiskiem były gwałtowne skoki ciśnienia (powyżej 15 hPa na kwadrans), co obniżało *Quality Score* pojedynczego pomiaru do poziomu 85,7%. Nierzadko towarzyszyły temu równie drastyczne skoki w zmierzonej wilgotności. Połączone błędy powodowały że ostateczny wynik wiarygodności takiego pomiaru wynosił 71,4%.

Ze względu na stabilność pogody w okresie testowania system nie wyłapał żadnych innych błędów, jednak podczas testowania działania programu pod względem wszystkich komunikatów za pomocą spreparowanych pomiarów stwierdzono poprawność wszystkich mechanizmów walidacji programu.

4.2 Zestawienie Ilościowe Zdiagnozowanych Błędów

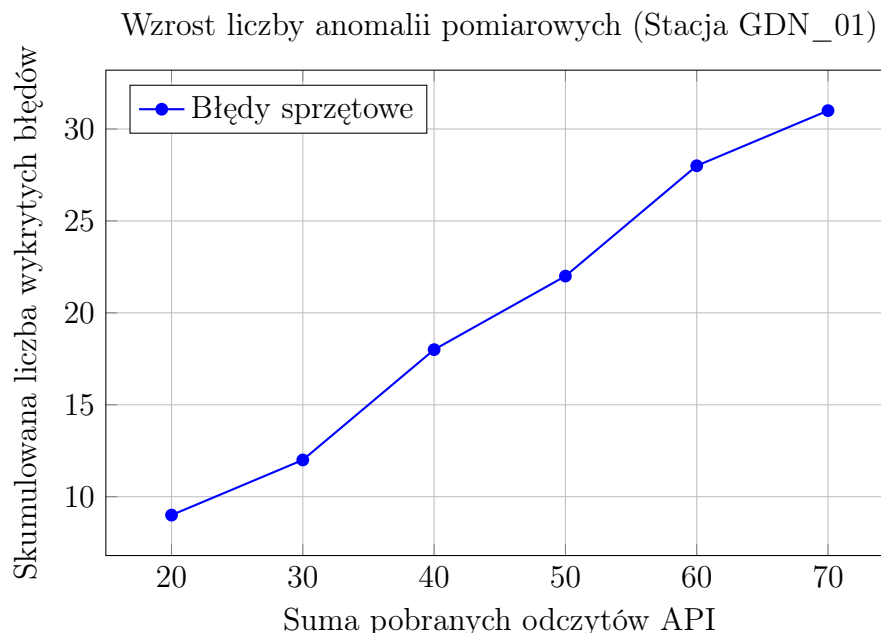
Zebrane błędy podzielono na dwa występujące rodzaje. Znacząca liczba błędów związanych z skokami ciśnienia (18), sugeruje że czujnik ciśnień w badanej stacji meteorologicznej GDN_01 może być uszkodzony. Ilość błędów związanych ze skokiem wilgoci jest dwukrotnie mniejsza (9), co może być niewystarczające do wyciągania wniosków o czujniku pomiarowym. Zaobserwowano również 5 pomiarów w których oba zmierzone parametry zostały oflagowane jako błędne.

Tabela 1: Częstotliwość występowania poszczególnych błędów sprzętowych na stacji GDN_01 (próba N=70).

Zdiagnozowana Anomalia	Liczba Wystąpień	Procent [N=70] (%)
Skok Pres: Różnica > 15 hPa	19	27,1%
Skok Hum: Różnica > 30%	8	11,4%
<i>W tym rekordy wykazujące oba błędy naraz:</i>	<i>5</i>	<i>7,1%</i>

4.3 Dynamika Przyrostu Błędów

Aby zilustrować częstotliwość występowania błędnych odczytów, przygotowano poniższy wykres. Ukazuje on skumulowaną sumę flagowanych przez system anomalii w zestawieniu z ogólną liczbą zebranych pakietów (dane z zapisów `report_GDN_01.jsonl`).



Rysunek 1: Wykres liniowy obrazujący przyrost błędnych odczytów.

5 Podsumowanie

Opracowany potok danych z powodzeniem realizuje zadanie separacji wiarygodnych pomiarów od odczytów zniekształconych. Zgromadzone przez system statystyki jednoznacznie wykazują, że znaczny udział usterek sprzętowych (stanowiących niemal połowę flagowanych odczytów pod koniec nocnej sesji) potwierdza absolutną konieczność stosowania mechanizmów kwarantanny. Przekazanie nieoczyszczonych danych bezpośrednio do systemów uczenia maszynowego lub narzędzi analitycznych drastycznie obniżyłoby wiarygodność wyników końcowych, przed czym wdrożone rozwiązanie skutecznie i w pełni automatycznie chroni.